

LAPORAN PRAKTIKUM 5

SINGLE LINKED LIST

STRUKTUR DATA



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Dr. Ir. Wahyudi, S. T., M. T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulisan laporan praktikum dengan judul “Single Linked List” ini dapat diselesaikan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah Praktikum Struktur Data.

Laporan ini akan membahas secara mendalam mengenai konsep, logik dasar, serta implementasi struktur data *Single Linked List* menggunakan bahasa pemrograman Java. Melalui pembahasan ini, diharapkan pembaca dapat memahami bagaimana data disimpan dan diakses secara dinamis menggunakan konsep node dan pointer, yang merupakan fondasi penting dalam manajemen memori dan pengembangan perangkat lunak..

Penulis menyadari bahwa masih terdapat berbagai kekurangan, baik dari segi penulisan maupun analisis kode program. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu maupun asisten praktikum sangat diharapkan demi perbaikan di masa mendatang. Semoga dokumentasi praktikum ini dapat memberikan manfaat tambahan bagi pembaca.

Padang, 5 Mei 2026

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	
2.1 Landasan Teori	3
2.2 Alat dan Bahan.....	3
2.3 Langkah Praktikum dan Kode Program	3
2.3.1 Kelas NodeSLL	3
2.3.2 Kelas TambahSLL.....	4
2.3.3 Kelas HapusSLL	7
2.3.4 Kelas PencarianSLL	9
BAB III PENUTUP	
3.1 Kesimpulan.....	12
3.2 Saran.....	12
DAFTAR PUSTAKA.....	13
TUGAS 5 PRAKTIKUM STRUKTUR DATA.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia ilmu komputer dan rekayasa perangkat lunak, pemahaman mengenai struktur data adalah hal yang sangat krusial. Struktur data menentukan bagaimana informasi disimpan, disusun, dan diakses di dalam memori komputer agar program dapat berjalan dengan efisien. Salah satu struktur data yang paling fundamental adalah array. Namun, array memiliki keterbatasan utama, yaitu sifatnya yang statis dan ukurannya yang harus ditentukan di awal. Hal ini seringkali menyebabkan pemborosan memori jika alokasi terlalu besar, atau justru terjadi overflow jika alokasi terlalu kecil.

Untuk mengatasi keterbatasan tersebut, diperkenalkanlah struktur data dinamis yang disebut *Linked List*. Salah satu bentuk dasar dari *Linked List* adalah *Single Linked List* (SLL). SLL memungkinkan penyimpanan data secara fleksibel di mana memori dialokasikan hanya saat dibutuhkan. Berbeda dengan array yang datanya bersebelahan secara fisik di memori, elemen-elemen pada SLL tersebar dan dihubungkan menggunakan referensi atau *pointer*. Pemahaman mengenai SLL tidak hanya penting untuk menyimpan data secara dinamis, tetapi juga menjadi dasar pijakan untuk mempelajari struktur data tingkat lanjut lainnya seperti *Tree* dan *Graph*.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum struktur data mengenai Single Linked List ini adalah sebagai berikut:

- 1) Memberikan pemahaman mendasar kepada mahasiswa mengenai konsep Single Linked List (SLL) beserta struktur pembentuk utamanya yaitu Node.
- 2) Melatih kemampuan mahasiswa dalam merancang dan menulis kode program Java untuk mengimplementasikan berbagai operasi dasar SLL, seperti penambahan data (di awal, akhir, dan posisi tertentu).

- 3) Membekali mahasiswa dengan keterampilan merancang algoritma penghapusan data (di awal, akhir, dan posisi tertentu) serta algoritma pencarian elemen pada SLL secara tepat.
- 4) Melatih kemampuan analisis logika dalam menelusuri (traversal) hubungan antar pointer tanpa memutus rantai data yang sudah ada.

1.3 Manfaat Praktikum

Melalui praktikum ini, mahasiswa diharapkan dapat memperoleh manfaat akademis dan teknis yang saling berkesinambungan. Secara akademis, mahasiswa dapat melihat secara langsung bentuk konkret dari teori manajemen memori dinamis yang telah diajarkan di kelas. Mahasiswa tidak lagi hanya membayangkan bagaimana pointer bekerja, melainkan langsung mempraktikkannya.

Secara teknis, kemampuan pemecahan masalah (problem-solving) mahasiswa akan meningkat pesat. Dengan membiasakan diri menulis kode program SLL, mahasiswa terlatih untuk berpikir sistematis, teliti dalam memanipulasi objek Java, dan lebih berhati-hati terhadap potensi masalah pemrograman seperti `NullPointerException`. Keterampilan ini akan sangat berguna ketika mahasiswa dihadapkan pada proyek pembuatan perangkat lunak berskala besar di kemudian hari.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Single Linked List (SLL) adalah sebuah koleksi elemen data linear yang disebut sebagai node atau simpul. Berbeda dengan array konvensional, penempatan setiap elemen SLL di dalam memori komputer tidak diatur secara bersebelahan. Sebagai gantinya, setiap node di dalam SLL dilengkapi dengan sebuah petunjuk (pointer/referensi) yang menyimpan alamat memori dari node berikutnya. Elemen pertama dari senarai ini disebut sebagai Head, sedangkan elemen terakhirnya akan menunjuk ke nilai Null, yang menandakan akhir dari sebuah antrean data.

Secara arsitektur, sebuah node tunggal pada SLL selalu terdiri atas dua bagian utama. Bagian pertama adalah Data Field, yang berfungsi untuk menyimpan informasi atau nilai aktual yang ingin disimpan, misalnya bilangan bulat (integer) atau teks (string). Bagian kedua adalah Next Pointer, yakni sebuah referensi yang bertugas mengaitkan node saat ini dengan node di depannya. Operasi dasar pada SLL meliputi Insertion (penyisipan data), Deletion (penghapusan data), Searching (pencarian), dan Traversal (penelusuran untuk menampilkan seluruh data). Sifat SLL yang hanya memiliki satu arah petunjuk (hanya bisa maju) menjadikannya sangat efisien untuk proses penambahan data secara dinamis.

2.2 Alat dan Bahan

- 1) Perangkat keras seperti komputer atau laptop yang memadai.
- 2) Perangkat lunak seperti sistem operasi pendukung misal Windows atau Linux, Java Development Kit (JDK) versi 8 ke atas, serta Integrated Development Environment (IDE) seperti Eclipse.

2.3 Langkah Praktikum dan Kode Program

2.3.1 Kelas NodeSLL

Langkah pertama adalah membuat pondasi, yaitu membuat kerangka untuk sebuah node. Kode berikut mendefinisikan objek tunggal penyusun SLL.

```

package pekan5_2511532023;

public class NodeSLL_2511532023 {
    // Node bagian data
    int data_2023;
    // Pointer ke node berikutnya
    NodeSLL_2511532023 next_2023;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511532023(int data_2023) {
        this.data_2023 = data_2023;
        this.next_2023 = null;
    }
}

```

Kode Program 2.3.1

Analisis Kode:

- `int data_2023;` : Ini adalah variabel untuk menyimpan nilai data aktual ke dalam memori. Bertipe integer.
- `NodeSLL_2511532023 next_2023;` : Ini adalah pointer. Variabel ini bertipe objek dari kelas itu sendiri, fungsinya sebagai penghubung ke node berikutnya.
- `public NodeSLL_2511532023(...)` : Ini merupakan konstruktor. Ketika kita membuat node baru, kita wajib memasukkan data. Secara otomatis (default), pointer `next_2023` diatur menjadi null karena node baru tersebut belum terhubung ke mana-mana.

2.3.2 Kelas TambahSLL

Langkah kedua adalah membuat logika algoritma untuk memasukkan node baru ke dalam rangkaian SLL, baik di awal, di akhir, maupun di tengah.

```

package pekan5_2511532023;

public class TambahSLL_2511532023 {
    public static NodeSLL_2511532023
insertAtFront_2023(NodeSLL_2511532023 head_2023, int value_2023) {
        NodeSLL_2511532023 new_node = new
NodeSLL_2511532023(value_2023);
        new_node.next_2023 = head_2023;
        return new_node;
    }
    // Fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511532023
insertAtEnd_2023(NodeSLL_2511532023 head_2023, int value_2023) {
        // Buat sebuah node dengan sebuah nilai
        NodeSLL_2511532023 newNode = new
NodeSLL_2511532023(value_2023);
        // Jika list kosong maka node jadi head

```

```

        if (head_2023 == null) {
            return newNode;
        }
        // Simpan head ke variabel sementara
        NodeSLL_2511532023 last_2023 = head_2023;
        // Telusuri ke node akhir
        while (last_2023.next_2023 != null) {
            last_2023 = last_2023.next_2023;
        }
        // Ubah pointer
        last_2023.next_2023 = newNode;
        return head_2023;
    }

    static NodeSLL_2511532023 GetNode_2023(int data_2023) {
        return new NodeSLL_2511532023(data_2023);
    }

    static NodeSLL_2511532023 insertPos(NodeSLL_2511532023
headNode_2023, int position_2023, int value_2023) {
        NodeSLL_2511532023 head_2023 = headNode_2023;
        if (position_2023 < 1)
            System.out.print("Invalid position");
        if (position_2023 == 1) {
            NodeSLL_2511532023 new_node = new
NodeSLL_2511532023(value_2023);
            new_node.next_2023 = head_2023;
            return new_node;
        } else {
            while (position_2023-- != 0) {
                if (position_2023 == 1) {
                    NodeSLL_2511532023 newNode =
GetNode_2023(value_2023);
                    newNode.next_2023 =
headNode_2023.next_2023;
                    headNode_2023.next_2023 = newNode;
                    break;
                }
                headNode_2023 = headNode_2023.next_2023;
            }
        }
        if (position_2023 != 1)
            System.out.print("Posisi di luar jangkauan");
        } return head_2023; }
    public static void printList(NodeSLL_2511532023 head_2023)
{
        NodeSLL_2511532023 curr = head_2023;
        while (curr.next_2023 != null) {
            System.out.print(curr.data_2023 + "-->");
            curr = curr.next_2023;
        }
        if (curr.next_2023 == null) {
            System.out.print(curr.data_2023);
            System.out.println(); }
    }

    public static void main(String[] args) {
        // Buat linked list 2->3->5->6
        NodeSLL_2511532023 head_2023 = new
NodeSLL_2511532023(2);
        head_2023.next_2023 = new NodeSLL_2511532023(3);
    }
}

```



```

        head_2023.next_2023.next_2023 = new
NodeSLL_2511532023(5);
        head_2023.next_2023.next_2023.next_2023 = new
NodeSLL_2511532023(6);
        // Cetak list asli
        System.out.print("Senarai berantai awal : ");
        printList(head_2023);
        // Tambahkan node baru di depan
        System.out.print("Tambah 1 simpul di depan : ");
        int data_2023 = 1;
        head_2023 = insertAtFront_2023(head_2023,
data_2023);

        // Cetak update list
        printList(head_2023);
        // Tambahkan node baru di belakang
        System.out.print("Tambah 1 simpul ke belakang : ");
        int data2 = 7;
        head_2023 = insertAtEnd_2023(head_2023, data2);
        // Cetak update list
        printList(head_2023);
        System.out.print("Tambah 1 simpul ke data 4 : ");
        int data3 = 4;
        int pos = 4;
        head_2023 = insertPos(head_2023, pos, data3);
        // Cetak update list
        printList(head_2023);
    }
}

```

Kode Program 2.3.2

Analisis Kode:

- insertAtFront: `new_node.next_2023 = head_2023`; menginstruksikan node baru untuk menunjuk ke arah Head (elemen paling depan) yang lama. Lalu, node baru tersebut dikembalikan (return) untuk dijadikan sebagai Head yang baru. Ini sangat cepat karena tidak perlu memeriksa elemen lain.
- insertAtEnd: Pertama, kode memeriksa apakah antrean masih kosong (`head == null`). Jika tidak kosong, program menggunakan perulangan while (`last_2023.next_2023 != null`) untuk terus melangkah mencari node mana yang menunjuk ke null (node terakhir). Setelah ditemukan, pointer node terakhir tersebut diubah agar menunjuk ke node yang baru dimasukkan.

Output:

```
Console X
<terminated> TambahSLL_2511532023 [Java Application] C:\Users\ACER\p
Senarai berantai awal : 2-->3-->5-->6
Tambah 1 simpul di depan : 1-->2-->3-->5-->6
Tambah 1 simpul ke belakang : 1-->2-->3-->5-->6-->7
Tambah 1 simpul ke data 4 : 1-->2-->3-->4-->5-->6-->7
```

2.3.3 Kelas HapusSLL

Langkah ketiga adalah menghapus node dari rantai tanpa merusak susunan rantai yang tersisa. Pada Java, data yang terputus ikatannya (tidak ditunjuk pointer mana pun) akan otomatis dibersihkan oleh sistem Garbage Collector.

```
package pekan5_2511532023;

public class HapusSLL_2511532023 {
    // Fungsi untuk menghapus head
    public static NodeSLL_2511532023
deleteHead_2023(NodeSLL_2511532023 head_2023) {
    // Jika SLL kosong
    if (head_2023 == null)
        return null;
    // Pindahkan head ke node berikutnya
    head_2023 = head_2023.next_2023;
    // Return head baru
    return head_2023; }
    // Fungsi menghapus node terakhir SLL
    public static NodeSLL_2511532023
removeLastNode_2023(NodeSLL_2511532023 head_2023) {
    // Jika list kosong, return null
    if (head_2023 == null) {
        return null;
    }
    // Jika list satu node, hapus node dan return null
    if (head_2023.next_2023 == null) {
        return null;
    }
    // Temukan node terakhir ke dua
    NodeSLL_2511532023 secondLast = head_2023;
    while (secondLast.next_2023.next_2023 != null) {
        secondLast = secondLast.next_2023;
    }
    // Hapus node terakhir
    secondLast.next_2023 = null;
    return head_2023; }
    // Fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511532023
deleteNode_2023(NodeSLL_2511532023 head_2023, int position_2023) {
```

```

NodeSLL_2511532023 temp = head_2023;
NodeSLL_2511532023 prev = null;
// Jika linked list null
if (temp == null)
    return head_2023;
// Kasus 1: head dihapus
if (position_2023 == 1) {
    head_2023 = temp.next_2023;
    return head_2023; }
// Kasus 2: menghapus node di tengah
// Telusuri ke node yang dihapus
for (int i = 1; temp != null && i < position_2023;
i++) {
    prev = temp;
    temp = temp.next_2023; }
// Jika ditemukan, hapus node
if (temp != null) {
    prev.next_2023 = temp.next_2023;
} else {
    System.out.println("Data tidak ada"); }
return head_2023; }
// Fungsi mencetak SLL
public static void printList (NodeSLL_2511532023
head_2023) {
    NodeSLL_2511532023 curr_2023 = head_2023;
    while (curr_2023.next_2023 != null) {
        System.out.print(curr_2023.data_2023 +
"-->");
        curr_2023 = curr_2023.next_2023;
        if (curr_2023.next_2023 == null) {
            System.out.print(curr_2023.data_2023);
            System.out.println(); }
        }
    }
// Kelas main
public static void main(String[] args) {
    // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 ->
null
NodeSLL_2511532023 head_2023 = new
NodeSLL_2511532023(1);
head_2023.next_2023 = new
NodeSLL_2511532023(2);
head_2023.next_2023.next_2023 = new
NodeSLL_2511532023(3);
head_2023.next_2023.next_2023.next_2023 = new
NodeSLL_2511532023(4);
head_2023.next_2023.next_2023.next_2023.next_2023 = new
NodeSLL_2511532023(5);
head_2023.next_2023.next_2023.next_2023.next_2023.next_2023 = new
NodeSLL_2511532023(6);
// Cetak list awal
System.out.println("List awal : ");
printList(head_2023);
// Hapus head
head_2023 = deleteHead_2023(head_2023);

```

```

        System.out.println("List setelah head dihapus
: ");
        printList(head_2023);
        // Hapus node terakhir
        head_2023 = removeLastNode_2023(head_2023);
        System.out.println("List setelah simpul
terakhir di hapus : ");
        printList(head_2023);
        // Deleting node at position 2
        int position_2023 = 2;
        head_2023 = deleteNode_2023(head_2023,
position_2023);
        // Print list after deletion
        System.out.println("List setelah posisi 2
dihapus : ");
        printList(head_2023);
    }
}

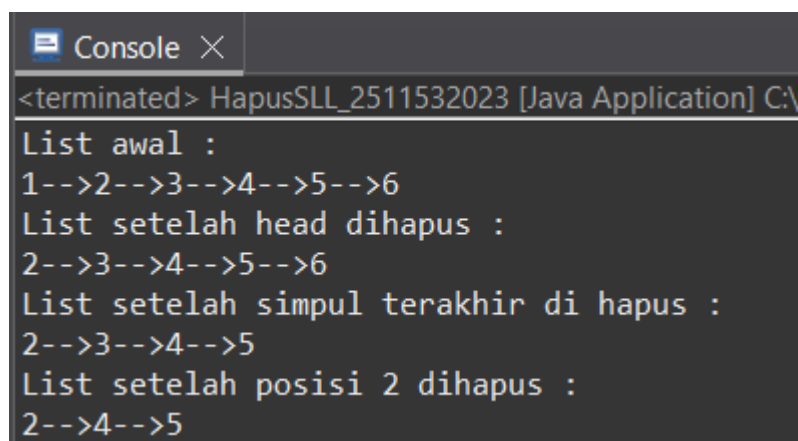
```

Kode Program 2.3.3

Analisis Kode:

- deleteHead: head_2023 = head_2023.next_2023; memindahkan label Head ke elemen nomor dua. Secara otomatis, elemen nomor satu yang lama akan dilupakan oleh program dan terhapus secara alami.
- removeLastNode: secondLast.next_2023.next_2023 != null adalah taktik untuk mencari node sebelum node terakhir (node kedua dari belakang). Setelah menemukannya, tali penghubungnya diputus dengan cara disetel menjadi null. Sehingga node yang paling ujung terlepas dari rangkaian.

Output:



```

Console X
<terminated> HapusSLL_2511532023 [Java Application] C:\
List awal :
1-->2-->3-->4-->5-->6
List setelah head dihapus :
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus :
2-->3-->4-->5
List setelah posisi 2 dihapus :
2-->4-->5

```

2.3.4 Kelas PencarianSLL

Langkah terakhir adalah membuat metode penelusuran (Traversal) untuk

mencetak semua isi antrian, serta mencari apakah suatu nilai spesifik ada di dalam SLL tersebut.

```
package pekan5_2511532023;

public class PencarianSLL_2511532023 {
    static boolean searchKey(NodeSLL_2511532023 head_2023, int
key_2023) {
        NodeSLL_2511532023 curr_2023 = head_2023;
        while (curr_2023 != null) {
            if (curr_2023.data_2023 == key_2023)
                return true;
            curr_2023 = curr_2023.next_2023; }
        return false; }
    public static void traversal(NodeSLL_2511532023 head_2023) {
        // Mulai dari head
        NodeSLL_2511532023 curr = head_2023;
        // Telusuri sampai pointer null
        while (curr != null) {
            System.out.print(" " + curr.data_2023);
            curr = curr.next_2023; }
        System.out.println(); }
    public static void main(String[] args) {
        NodeSLL_2511532023 head_2023 = new NodeSLL_2511532023(14);
        head_2023.next_2023 = new NodeSLL_2511532023(21);
        head_2023.next_2023.next_2023 = new NodeSLL_2511532023(13);
        head_2023.next_2023.next_2023.next_2023 = new
NodeSLL_2511532023(30);
        head_2023.next_2023.next_2023.next_2023.next_2023 = new
NodeSLL_2511532023(10);
        System.out.print("Penelusuran SLL : ");
        traversal(head_2023);
        // Data yang akan dicari
        int key_2023 = 30;
        System.out.print("Cari data " + key_2023 + " = ");
        if (searchKey(head_2023, key_2023))
            System.out.println("Ketemu");
        else
            System.out.println("Tidak ada");
    }
}
```

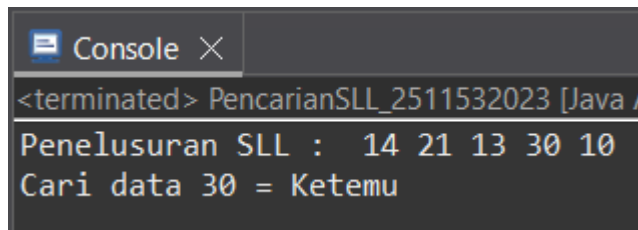
Kode Program 2.3.4

Analisis Kode:

- Membuat objek sementara bernama curr_2023 dan meletakkannya di posisi awal (head_2023).
- Perulangan while (curr_2023 != null) berfungsi untuk melangkah dari satu node ke node lain.
- Logika if (curr_2023.data_2023 == key_2023) mencocokkan data pada node saat ini dengan angka target. Jika cocok, kembalikan nilai true (ketemu).

Jika melangkah sampai ujung (null) dan tidak ketemu, fungsi akan mengembalikan false.

Output:



```
Console X
<terminated> PencarianSLL_2511532023 [Java A
Penelusuran SLL : 14 21 13 30 10
Cari data 30 = Ketemu
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa Single Linked List adalah struktur data yang sangat fleksibel dan dinamis dalam mengelola alokasi memori. Berbeda dengan array, SLL memungkinkan pengguna untuk menambah atau mengurangi kapasitas penyimpanannya di tengah jalannya program tanpa harus mengkhawatirkan batas maksimal memori. Komponen utama dari SLL adalah Node yang berisi data, serta sebuah pointer tunggal yang menghubungkan antrean dari elemen terdepan (Head) hingga elemen paling akhir yang ditutup dengan null.

Dari segi komputasi, operasi penambahan dan penghapusan data di bagian depan (Head) SLL berjalan sangat cepat. Namun, untuk operasi pencarian data atau modifikasi data di bagian akhir, program harus melakukan penelusuran berulang (Traversal) mulai dari depan hingga titik akhir satu per satu. Oleh sebab itu, pemahaman yang kuat mengenai alur gerak pointer sangat diperlukan agar tidak terjadi eror di mana rantai penunjuk terputus, yang dapat menyebabkan data hilang atau kebocoran memori.

3.2 Saran

Dalam mempraktikkan pengkodean SLL di masa mendatang, sangat disarankan agar lebih teliti dan berhati-hati dalam menempatkan urutan pergantian pointer. Sebuah kesalahan kecil, seperti menghubungkan node tanpa mengamankan node berikutnya terlebih dahulu, dapat memutus rantai data secara permanen. Penggambaran bagan struktur menggunakan coretan di kertas sebelum mengetik kode dapat sangat membantu merapikan logika berpikir.

DAFTAR PUSTAKA

- [1] Academia.edu, “LAPORAN PRAKTIKUM VI,” 2020. [Daring]. Tersedia pada: https://www.academia.edu/31899450/LAPORAN_PRAKTIKUM_VI_docx. [Diakses: 06-Mei-2026].
- [2] Studocu, “Laporan Praktikum Modul 5 – Struktur Data,” UIN Sunan Gunung Djati. [Daring]. Tersedia pada: <https://www.studocu.id/id/document/universitas-islam-negeri-sunan-gunung-djati/praktikum-struktur-data/laporan-praktikum-modul-5/45776809>. [Diakses: 07-Mei-2026].
- [3] Scrtibd, “Laporan Praktikum SLL 1,” [Daring]. Tersedia pada: <https://www.scribd.com/document/497439589/Laporan-Praktikum-SLL-1>. [Diakses: 07-Mei-2026].
- [4] Studocu, “Laporan Praktikum VI – Fisip,” Universitas Islam Blitar. [Daring]. Tersedia pada: <https://www.studocu.id/id/document/universitas-islam-blitar/fakultas-ilmu-sosial-dan-ilmu-politik/laporan-praktikum-vi-docx/62254982>. [Diakses: 07-Mei-2026].

TUGAS 5 PRAKTIKUM STRUKTUR DATA

1) Soal

Mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2) Jawab

- Kelas ADT Pasien_2511532023

```
package pekan5_2511532023;

public class Pasien_2511532023 {
    // Atribut
    private String namaPasien_2023;
    private String keluhan_2023;
    private int nomorAntrian_2023;

    // Pointer
    private Pasien_2511532023 next_2023;

    // Konstruktor
    public Pasien_2511532023(String namaPasien_2023, String
keluhan_2023, int nomorAntrian_2023) {
        this.namaPasien_2023 = namaPasien_2023;
        this.keluhan_2023 = keluhan_2023;
        this.nomorAntrian_2023 = nomorAntrian_2023;
        this.next_2023 = null; // Default null karena belum ada antrian
di belakangnya
    }

    // Setter
    public String getNamaPasien_2023() {
        return namaPasien_2023;
    }
    public String getKeluhan_2023() {
        return keluhan_2023;
    }
    public int getNomorAntrian_2023() {
        return nomorAntrian_2023;
    }
    public Pasien_2511532023 getNext_2023() {
        return next_2023;
    }
}
```

```
// Getter
public void setNamaPasien_2023(String namaPasien_2023) {
    this.namaPasien_2023 = namaPasien_2023;
}
public void setKeluhan_2023(String keluhan_2023) {
    this.keluhan_2023 = keluhan_2023;
}
public void setNomorAntrian_2023(int nomorAntrian_2023) {
    this.nomorAntrian_2023 = nomorAntrian_2023;
}
public void setNext_2023(Pasien_2511532023 next_2023) {
    this.next_2023 = next_2023;
}
}
```

Kelas ini bertugas sebagai cetakan untuk satu Node. Ibaratnya kelas ini sebagai satu lembar tiket antrian fisik yang dipegang oleh pasien.

- Atribut Data: Di dalam tiket tersebut, tercatat informasi spesifik pasien seperti Nama Pasien , Keluhan / Penyakit , dan Nomor Antrian
- Pointer next: Ini adalah bagian terpenting dari konsep Linked List. Atribut next_2023 berfungsi sebagai penunjuk ke arah pasien yang mengantri tepat di belakangnya. Jika nilai next adalah null, berarti orang tersebut adalah orang terakhir di barisan dan tidak ada lagi yang mengantri di belakangnya.
- Konstruktor: Saat pasien baru datang, petugas membuat tiket baru (memanggil constructor) dan mengisi data pasien tersebut. Karena pasien baru datang, otomatis dia berada di posisi paling belakang, sehingga penunjuk next_2023 secara bawaan (default) diatur menjadi null.
- Getter & Setter: Method ini digunakan untuk membaca data (selektor) dan mengubah data (mutator) pada pasien tersebut secara aman tanpa merusak struktur program.
- Kelas Driver RumahSakit_2511532023

```
package pekan5_2511532023;
import java.util.Scanner;
```

```

public class RumahSakit_2511532023 {
    private Pasien_2511532023 head_2023; // Head linked list
    private int counterAntrian_2023; // Counter yang otomatis menambah
    nomor antrian berikutnya
    private int jumlahPasien_2023; // Menyimpan jumlah total pasien di
    antrian saat ini

    public RumahSakit_2511532023() {
        this.head_2023 = null;
        this.counterAntrian_2023 = 1;
        this.jumlahPasien_2023 = 0;
    }

    // 1. Daftar Pasien
    public void daftarkanPasien_2023(String namaPasien_2023, String
    keluhan_2023) {
        Pasien_2511532023 pasienBaru_2023 = new
        Pasien_2511532023(namaPasien_2023, keluhan_2023, counterAntrian_2023);

        // Jika list kosong, node baru langsung jadi head
        if (head_2023 == null) {
            head_2023 = pasienBaru_2023;
        } else {
            // Traverse sampai ke node paling akhir (tail)
            Pasien_2511532023 temp_2023 = head_2023;
            while (temp_2023.getNext_2023() != null) {
                temp_2023 = temp_2023.getNext_2023();
            }
            // Menyambungkan node terakhir dengan node baru
            temp_2023.setNext_2023(pasienBaru_2023);
        }

        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian:
        " + counterAntrian_2023);
        counterAntrian_2023++; // Otomatis menambah nomor antrian untuk
        pasien berikutnya
        jumlahPasien_2023++;
    }

    // 2. Panggil Pasien
    public void panggilPasien_2023() {
        // Saat list kosong
        if (head_2023 == null) {
            System.out.println("Antrian kosong. Tidak ada pasien yang
            dapat dipanggil.");
            return;
        }

        // Mengambil data head sebelum dihapus
        Pasien_2511532023 pasienDipanggil_2023 = head_2023;

        System.out.println("\n=== Memanggil Pasien ===");
        System.out.println("Nomor Antrian : " +
        pasienDipanggil_2023.getNomorAntrian_2023());
        System.out.println("Nama Pasien : " +
        pasienDipanggil_2023.getNamaPasien_2023());
    }
}

```

```

        System.out.println("Keluhan          : " +
pasienDipanggil_2023.getKeluhan_2023());

        // Head digeser ke node berikutnya
        head_2023 = head_2023.getNext_2023();
        jumlahPasien_2023--;
    }

    // 3. Tampilkan Antrian
    public void tampilkanAntrian_2023() {
        if (head_2023 == null) {
            System.out.println("Antrian saat ini kosong.");
            return;
        }
        System.out.println("\n=== Daftar Antrian Pasien ===");
        Pasien_2511532023 temp_2023 = head_2023;
        int posisi_2023 = 1;

        // Menelusuri list dari head sampai null
        while (temp_2023 != null) {
            System.out.println("Posisi " + posisi_2023 + " | No.
Antrian: " + temp_2023.getNomorAntrian_2023() +
                                " | Nama: " +
temp_2023.getNamaPasien_2023() +
                                " | Keluhan: " +
temp_2023.getKeluhan_2023());
            temp_2023 = temp_2023.getNext_2023();
            posisi_2023++;
        }
    }

    // 4. Cari Pasien
    public void cariPasien_2023(String cariNama_2023) {
        if (head_2023 == null) {
            System.out.println("Antrian kosong. Pencarian dibatalkan.");
            return;
        }
        Pasien_2511532023 temp_2023 = head_2023;
        boolean ditemukan_2023 = false;
        int posisi_2023 = 1;

        while (temp_2023 != null) {
            if
(temp_2023.getNamaPasien_2023().equalsIgnoreCase(cariNama_2023)) {
                System.out.println("\nPasien Ditemukan!");
                System.out.println("Nama Pasien    : " +
temp_2023.getNamaPasien_2023());
                System.out.println("Nomor Antrian : " +
temp_2023.getNomorAntrian_2023());
                System.out.println("Keluhan       : " +
temp_2023.getKeluhan_2023());
                System.out.println("Posisi Antrian: " + posisi_2023);
                ditemukan_2023 = true;
                break;
            }
            temp_2023 = temp_2023.getNext_2023();
            posisi_2023++;
        }
    }

```

```

    }
    if (!ditemukan_2023) {
        System.out.println("Pasien dengan nama '" + cariNama_2023 +
        "' tidak ditemukan dalam antrian.");
    }
}

// 5. Cek Status ANtrian
public void cekStatusAntrian_2023() {
    if (head_2023 == null) {
        System.out.println("Status Antrian: Kosong (0 pasien).");
    } else {
        System.out.println("\n=== Status Antrian ===");
        System.out.println("Total Pasien Menunggu: " +
        jumlahPasien_2023);
        System.out.println("Pasien Terdepan:");
        System.out.println("- Nama      : " +
        head_2023.getNamaPasien_2023());
        System.out.println("- Keluhan : " +
        head_2023.getKeluhan_2023());
        System.out.println("- No. Urut: " +
        head_2023.getNomorAntrian_2023());
    }
}

// Main method
public static void main(String[] args) {
    Scanner sc_2023 = new Scanner(System.in);
    RumahSakit_2511532023 rs_2023 = new RumahSakit_2511532023();
    int pilihan_2023;

    do {
        System.out.println("\nAntrian Rumah Sakit NIM: 2511532023");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");

        pilihan_2023 = sc_2023.nextInt();
        sc_2023.nextLine();
        switch (pilihan_2023) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_2023 = sc_2023.nextLine();
                System.out.print("Masukkan Keluhan : ");
                String keluhan_2023 = sc_2023.nextLine();
                rs_2023.daftarkanPasien_2023(nama_2023,
                keluhan_2023);
                break;
            case 2:
                rs_2023.panggilPasien_2023();
                break;
            case 3:
                rs_2023.tampilkanAntrian_2023();

```

```

        break;
    case 4:
        System.out.print("Masukkan Nama Pasien yang dicari:
");
        String cari_2023 = sc_2023.nextLine();
        rs_2023.cariPasien_2023(cari_2023);
        break;
    case 5:
        rs_2023.cekStatusAntrian_2023();
        break;
    case 6:
        System.out.println("Terima kasih. Program
selesai.");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
} while (pilihan_2023 != 6);

sc_2023.close();
}

```

Ibaratnya kelas ini Petugas Administrasi yang mengelola seluruh barisan antrian pasien. Kelas ini memiliki pointer utama bernama head_2023 yang tugasnya hanya satu: selalu menunjuk siapa pasien yang ada di urutan paling depan.

- Daftar Pasien

Logika ini bertujuan untuk menambahkan node pasien baru di ujung akhir linked list.

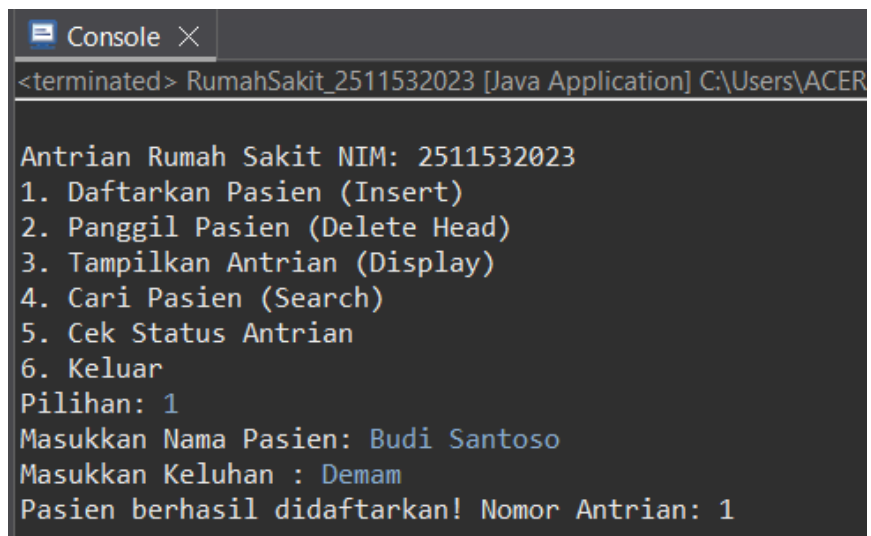
- 1) Pasien baru datang, node baru dibuat. NOmor antrian diberikan dari var counter otomatis.
- 2) Petugas mengecek apakah ruang tunggu kosong (head == null). Jika ya, pasien langsung menjadi orang pertama di antrian (head = pasienBaru).
- 3) Jika ruang tunggu sudah ada orang, petugas harus mencari siapa pasien yang ada di antrian paling belakang. Petugas menelusuri dari head, lalu melihat nilai next dari setiap pasien. Selama next belum bernilai null (while (temp.getNext() != null)), petugas terus bergeser ke belakang.

- 4) Setelah menemukan pasien dengan next bernilai null (pasien paling belakang), petugas menyambungkan pointer next pasien tersebut ke pasienBaru.
- Panggil Pasien
Logika ini menghapus node terdepan dan menampilkan data pasien yang dipanggil sesuai aturan FIFO.
 - 1) Petugas mengecek dulu apakah ada antrian. Jika list kosong (head == null), proses dibatalkan agar program tidak error.
 - 2) Data dari pasien yang ada di head diambil untuk diumumkan/ditampilkan.
 - 3) Barisan digeser dengan cara memindahkan status orang pertama (head) ke orang di belakangnya (head = head.getNext()). Pasien pertama tadi otomatis terlepas dari rantai dan dianggap sudah selesai dilayani.
 - Tampilkan Antrian
Logika ini menelusuri *linked list* dari urutan depan (head) hingga akhir (null) untuk menampilkan seluruh data.
 - 1) Petugas mulai dari pasien terdepan (temp = head).
 - 2) Petugas mencatat posisi (1, 2, 3...) dan mencetak data pasien tersebut.
 - 3) Petugas berpindah ke orang di belakangnya (temp = temp.getNext()). Proses ini diulang terus-menerus (looping) sampai petugas menemui nilai null (akhir dari antrian).
 - Cari Pasien
Logika ini melakukan pencarian pasien berdasarkan namanya.
 - 1) Sama seperti proses Display, petugas menelusuri barisan dari head ke belakang.
 - 2) Di setiap node, program mengecek apakah nama pasien cocok dengan nama yang diinputkan pengguna. Proses pengecekan menggunakan `.equalsIgnoreCase()` sehingga sistem tidak membedakan huruf kapital atau kecil secara ketat (Case-Insensitive).

- 3) Jika ditemukan, data dicetak dan proses pencarian dihentikan (break). Jika sampai akhir tidak ketemu, akan muncul peringatan.
- Cek Status Antrian
Ini adalah fungsi cepat untuk mengetahui rangkuman kondisi ruang tunggu. Petugas hanya perlu melihat variabel jumlahPasien untuk mengetahui sisa orang, lalu melihat secara langsung ke variabel head untuk mengetahui data pasien terdepan.
Dengan memisahkan antara entitas Pasien (ADT) dan sistem pengelolanya (Driver), program menjadi lebih rapi dan simulasi "rantai antrian" dapat bekerja secara dinamis tanpa batasan kapasitas (berbeda dengan tipe data Array yang ukurannya kaku).

Output:

- Daftarkan Pasien
 - Pasien Budi Santoso



```
<terminated> RumahSakit_2511532023 [Java Application] C:\Users\ACER
Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Budi Santoso
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1
```

- Pasien Andi


```
Console X
<terminated> RumahSakit_2511532023 [Java Application] C:\Users

Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Andi
Masukkan Keluhan : Flu
Pasien berhasil didaftarkan! Nomor Antrian: 2
```

- Tampilkan Antrian

```
Console X
<terminated> RumahSakit_2511532023 [Java Application] C:\Users\ACER\p2\pool\plug

Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

=== Daftar Antrian Pasien ===
Posisi 1 | No. Antrian: 1 | Nama: Budi Santoso | Keluhan: Demam
Posisi 2 | No. Antrian: 2 | Nama: Andi | Keluhan: Flu
```

- Panggil Pasien

```
Console X
<terminated> RumahSakit_2511532023 [Java Application] C:\Users\A

Antrian Rumah Sakit NIM: 2511532023|
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2

=== Memanggil Pasien ===
Nomor Antrian : 1
Nama Pasien   : Budi Santoso
Keluhan       : Demam
```

- Tampilkan Antrian

```
Console X
<terminated> RumahSakit_2511532023 [Java Application] C:\Users\ACER\p2\pool\

Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

=== Daftar Antrian Pasien ===
Posisi 1 | No. Antrian: 2 | Nama: Andi | Keluhan: Flu
```

- Cari Pasien
 - Pasien Budi

```

Console X
<terminated> RumahSakit_2511532023 [Java Application] C:\Users\ACER\p2\pool\plugin

Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: Budi Santoso
Pasien dengan nama 'Budi Santoso' tidak ditemukan dalam antrian.

```

- Pasien Andi

```

Console X
<terminated> RumahSakit_2511532023 [Java Application] C:\Use

Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: Andi

Pasien Ditemukan!
Nama Pasien   : Andi
Nomor Antrian : 2
Keluhan       : Flu
Posisi Antrian: 1

```

- Cek Status Antrian

```
Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
```

```
=== Status Antrian ===
Total Pasien Menunggu: 1
Pasien Terdepan:
- Nama      : Andi
- Keluhan   : Flu
- No. Urut  : 2
```

- Keluar

```
Antrian Rumah Sakit NIM: 2511532023
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih. Program selesai.
```